

Introduction to Databases

Lecture 2:

The relational model

Floris Geerts

What have we seen last time?

- Entity-Relationship modelling for describing a requirement analysis
 - Entity sets
 - Relationships between them
 - Constraints: keys, total/optional, cardinality
- Today we will cover:
 - The relational model
 - Implementing an Entity-Relationship model into the relational database model

Summary

- The relational model
 - Relation schema, tuple and relation instance
 - Database schema and instance
 - Constraints
 - Data Definition Language of SQL
- Translating ER-diagrams to the relational model

The Relational Model

- A *relation schema* is a set of pairs $\{ A_1:D_1, \dots, A_n:D_n \}$ with
 - A_i an *attribute name* ; that is: an element of a set of valid attribute names
 - D_i a domain; that is a possibly infinite set of values
- A *tuple* over schema $\{ A_1:D_1, \dots, A_n:D_n \}$ is a mapping from each attribute A_i to a value t_i in D_i

Example schema: $\{ \text{stud_id:INTEGER, name:VARCHAR, addr:VARCHAR} \}$

INTEGER: set of all integer numbers

VARCHAR: set of all character strings of variable length

Example tuple: $\{ \text{stud_id:10001, name:"Joske", addr:"Tramezantlei 122"} \}$

The Relational Model

- A *relation instance* over schema $\{ A_1:D_1, \dots, A_n:D_n \}$ is a finite set of tuples over this schema

Example relation instance:

stud_id	name	addr
10001	Joske Vermeulen	Tramezantlei 122, Schoten
10002	Dave Bridges	5th Avenue 5678, New York
10003	Davina Jones	5th Avenue 5678, NY

The Relational Model

- A *database schema* consists of:
 - A finite set of relation name-relation schema pairs;
 - A set of *integrity constraints* ; that is: a condition on the relation schemas that restricts the data that can be stored in the database
- A *database instance* of a database schema consists in a relation instance for each of the relations in the database that together satisfy the integrity constraints

** Theoretically speaking, hence an integrity constraint could be defined as a (possibly infinite) set of allowable database instances*

The Relational Model

Example database scheme:

Student: { stud_id:INTEGER, name:VARCHAR, addr:VARCHAR }

Course: { course_id:INTEGER, name:VARCHAR, lect_id:INTEGER }

Follows: { stud_id:INTEGER, course_id:INTEGER, grade:INTEGER }

Lecturer: { lect_id :INTEGER, name:VARCHAR }

Integrity constraints:

stud_id is a key in relation Student, course_id in Course, the pair (stud_id,course_id) in Follows, and lect_id in Lecturer

For every tuple t of Course, there exists a tuple in lecturer with key t.lect_id

For every tuple t of Follows, there a tuple in Student and a tuple in Course with key t.stud_id and t.course_id, respectively.

The Relational Model

- **Example database instance:**

Student

stud_id	name	addr
10001	Joske Vermeulen	Tramezantlei 122, Schoten
10002	Dave Bridges	5th Avenue 5678, New York
10003	Davina Jones	5th Avenue 5678, NY

Course

course_id	name	lect_id
1111	Intro to DB	1234

Lecturer

lect_id	name
1234	F. Geerts
2345	G. Bergmans

Follows

course_id	Stud_id
1111	10001
1111	10002

The Relational Model

- **NOT an instance:**

Student

stud_id	name	addr
10001	Joske Vermeulen	Tramezantlei 122, Schoten
10001	Dave Bridges	5th Avenue 5678, New York
10003	Davina Jones	5th Avenue 5678, NY

Course

course_id	name	lect_id
1111	Intro to DB	1234

Lecturer

lect_id	name
1234	F. Geerts
2345	G. Bergmans

Follows

course_id	Stud_id
1112	10001
1111	10005

The Relational Model: Notational Conventions

- We will use the notation $R(A_1:D_1, \dots, A_n:D_n)$ to denote that a relation instance R has schema $\{ A_1:D_1, \dots, A_n:D_n \}$.
 - We will often omit the domains from this notation and refer to there domain of A_i by $\text{dom}(A_i)$
- A tuple over relation $R(A_1, \dots, A_n)$ mapping A_i to t_i in $\text{dom}(A_i)$ may be denoted $(t_1, \dots, t_n)$.
- When there is confusion between two relations R and S sharing an attribute name A , we will use $R.A$ and $S.A$ to disambiguate.

Integrity Constraints

- The restriction that a tuple must map attributes to a value in their domain, will be called a *domain constraint*.
- There are two other, very common constraints:
 - Primary key constraints
 - Foreign key dependencies
- Later on we will see how we can define other, ad-hoc constraints using CHECKS and triggers

Key Constraint

- Let $R(A_1, \dots, A_n)$ be a relation scheme.
- A *key constraint* for R is a statement of the form:
 K is a key for relation R
(K is a subset of $\{ A_1, \dots, A_n \}$)
- A relation instance of this schema is *valid* for this constraint if for every tuple t in the instance, the combination of values for the attributes in K is unique
- A database designer can mark one of the keys of a relation as the *primary key* of that relation

Example Key Constraint

Constraint: $\{A,B\}$ is a key for relation $R(A,B,C)$

A	B	C
1	2	4
1	3	4
2	3	5

Valid instance

A	B	C
1	2	3
1	2	4
2	5	7

Invalid instance

Foreign Key Constraint

- Let $R(A_1, \dots, A_n)$ and $S(B_1, \dots, B_m)$ be relation schemes, and let S_{PK} be a **primary key** for S .
- A **foreign key constraint** is a statement of the form:
The set of attributes K of R form a foreign key that refers to S
(K is a subset of $\{A_1, \dots, A_n\}$ of cardinality $|S_{PK}|$)
- A database instance with relations R and S is **valid** for this constraint if for every tuple t in the instance of R , the combination of values for the attributes in K appears as a primary key in the instance of S

Foreign Key: Example

- { lect_id } is the primary key of relation Lecturer
- Constraint: Course.lect_id is a foreign key in Course that refers to Lecturer

Course

course_id	name	lect_id
1111	Intro to DB	1234

Lecturer

lect_id	name
1234	T. Calders
2345	G. Bergmans

Valid instance

Course

course_id	name	lect_id
1111	Intro to DB	1234

Lecturer

lect_id	name
2345	G. Bergmans

Invalid instance

Foreign Key: Example

- { course_id, stud_id } is primary key in Follows
- { course_id } is primary key in Course
- { course_id } is a Foreign key in Follows that refers to Course (similar for stud_id).

Student

stud_id	name	addr
10001	Joske Vermeulen	Tramezantlei 122, Schoten
10002	Dave Bridges	5th Avenue 5678, New York
10003	Davina Jones	5th Avenue 5678, NY

Course

course_id	name	lect_id
1111	Intro to DB	1234

Follows

course_id	Stud_id
1111	10001
1111	10002

Implementations of Relational Databases

- There exist multiple implementations of relational databases
 - Oracle, MySQL, PostgreSQL, MS SQL Server, DB II, MonetDB, ...
- Most of these implementations support the Structured Query Language (SQL)
 - **Data Definition Language** (DDL) that allows to define schemas and constraints
 - Insert and Delete statements to add and delete data in the tables
 - A **query language** for complex queries on the data combining multiple tables
- We will now see the DDL of SQL

Creating a Database

CREATE DATABASE University;

End SQL
statements with ;

Name of the
database

- Usually not everyone can create a database
 - Need the right permissions

USE University;

- Execute the next commands on the database with name “University”

Creating Tables in the Database

See http://www.w3schools.com/sql/sql_create_table.asp for create table statements. See also following topics.

```
CREATE TABLE Persons  
(  
  P_Id int,  
  LastName varchar(255),  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255)  
);
```

The diagram illustrates the components of the SQL `CREATE TABLE` statement. It shows the full statement with callouts pointing to specific parts:
- A callout labeled "Name of the table" points to `Persons`.
- A callout labeled "Attribute name" points to `P_Id`.
- A callout labeled "Attribute domain / type" points to `varchar(255)` in the `LastName` definition.

Creating Tables in the Database

- There are numerous predefined types
 - May differ from system to system, though
 - Good overview: http://www.w3schools.com/sql/sql_datatypes.asp
- Some of the more common types:

• int	integer
• varchar(n)	variable length string of maximum length n
• char(n)	fixed length string of length n
• real	real number
• date, time, datetime	date, time, and both a date and a time

Defining Primary Keys

```
CREATE TABLE Persons
```

```
(
```

```
  P_Id int NOT NULL UNIQUE,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255),
```

```
  PRIMARY KEY (P_Id)
```

```
);
```

There cannot
be two tuples
with the same
P_Id

Attribute MUST be present
in every tuple.
Mandatory for primary key
attributes

P_Id int UNIQUE would
simply specify **candidate
key** (not primary key)

Could also
be multiple
attributes

P_Id	LN	FN	Address	City
1	a	a	x	2
2	b	Null	y	2
3	b	c	z	3

Defining Foreign Keys

```
CREATE TABLE Orders  
(
```

```
  O_Id int NOT NULL,  
  OrderNo int NOT NULL,  
  P_Id int,  
  PRIMARY KEY (O_Id),
```

```
  FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
```

```
);
```

PRIMARY KEY (O_Id) already
implies that O_Id NOT NULL
and UNIQUE

Does not have to be the
same attribute name. But
number and types need to
match

Could also be
multiple attributes

If same Attributes then P_Id INT
REFERENCES Persons ok as well

Enforces that every Orders.P_Id occurs in the Persons table in the
column P_Id.

Exercise

- Consider the following tables:
 - Medicine(mcode, mname, mdescription)
 - Patient(SSN, name, address)
 - Doctor(licenceNr, name, specialization)
- Write the SQL DDL statements to create a table to hold prescriptions
 - Contains a medicine; quantity and mode of use are recorded
 - Is for a patient
 - Has a date and a unique barcode
 - There always has to be exactly one doctor for the prescription

```
CREATE TABLE Medicine (  
    mcode INT PRIMARY KEY,  
    mname VARCHAR,  
    mdescription VARCHAR);
```

```
CREATE TABLE Patient (  
    SSN INT PRIMARY KEY,  
    name VARCHAR,  
    address VARCHAR);
```

```
CREATE TABLE Doctor (  
    licenceNR INT PRIMARY KEY,  
    name VARCHAR,  
    specialization VARCHAR);
```

```
CREATE TABLE Prescription(  
    barcode INT PRIMARY KEY,  
    dateOfPrescription DATE,  
    doctorID INT NOT NULL REFERENCES Doctor(licenceNR),  
    patient INT REFERENCES Patient(SSN),  
    medicine INT REFERENCES Medicine(mcode),  
    quantity INT,  
    mou VARCHAR
```


Summary

- The relational model
 - Relation schema, tuple and relation instance
 - Database schema and instance
 - Constraints
 - Data Definition Language of SQL
- Translating ER-diagrams to the relational model

Translation: Notations

- As the ER diagrams do not contain types, we will omit definition of domains
 - Relation schema: $R(A,B,C)$
- We will underline primary keys
- We will explicitly mention foreign key constraints

Student(stud_id, name, addr)

Course(course_id, name, lect_id)

Follows(stud_id, course_id, grade)

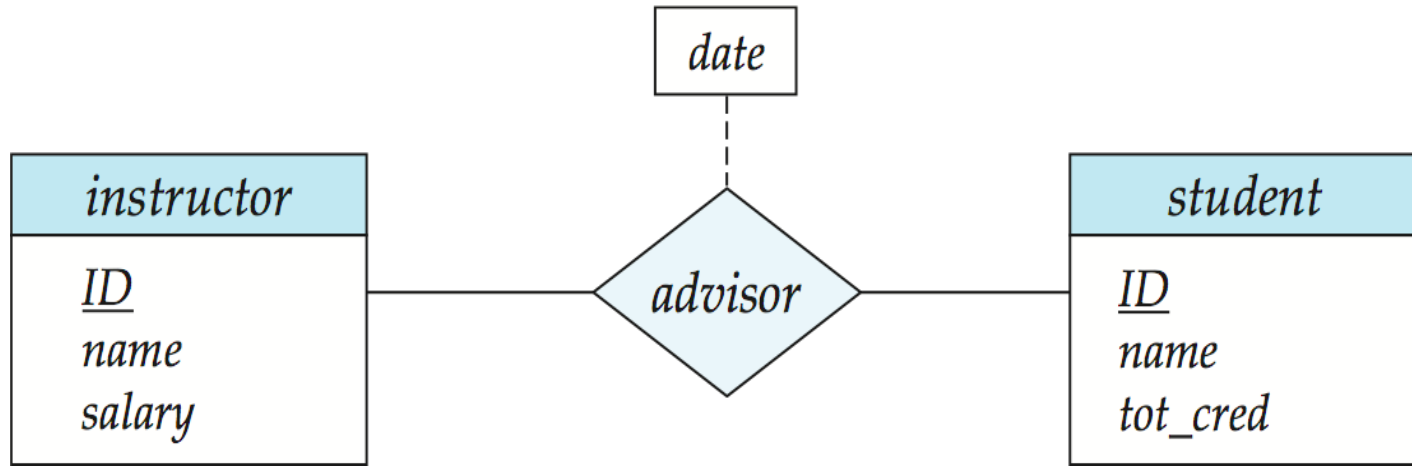
lect_id FK ref. Lecturer

stud_id FK ref. Student

course_id FK ref. Course

Lecturer(lect_id, name)

Translation: Examples



- instructor(iID, name, salary)
- student(sID, name, tot_cred)
- advisor(iID, sID, date)

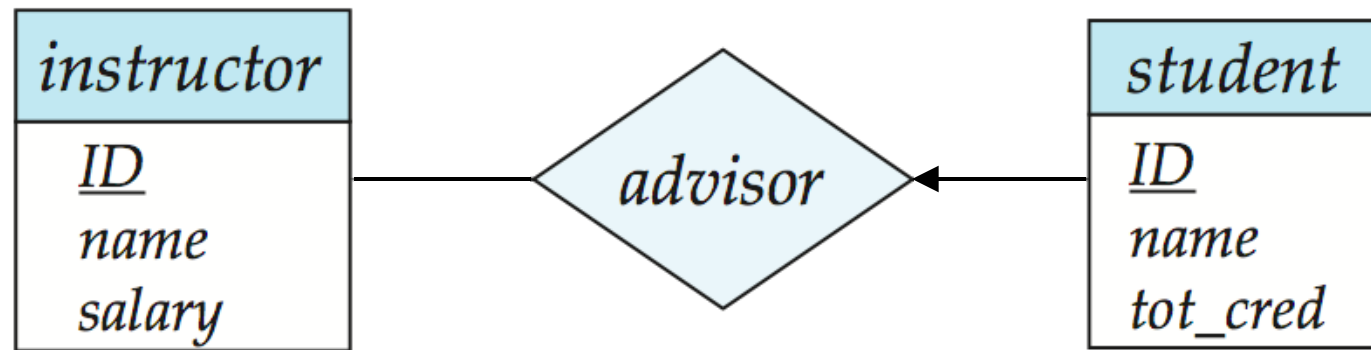
iID FK ref. instructor ; sID FK ref. student

Not ok:

- advisor(iID, sID, date): at most one student/advisor
- Advisor(iID, sID, date): at most one advisor/student

Translation: Examples

- One to many, solution without NULLs

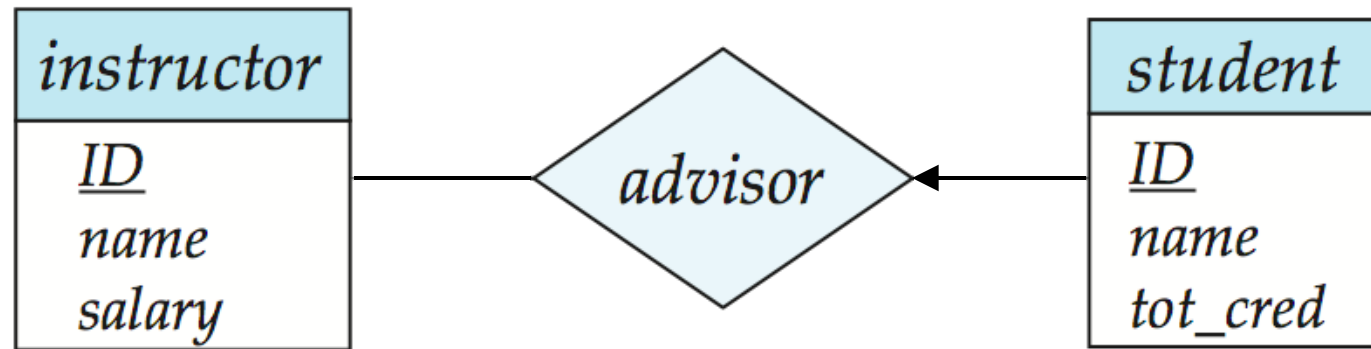


- `instructor(iID,name,salary)`
- `student(sID,name,tot_cred)`
- `advisor(sID, iID)` `iID` FK ref. instructor ; `sID` FK ref. student

If `iID` NOT NULL then ensure that every tuple in advisor relation has an advisor.

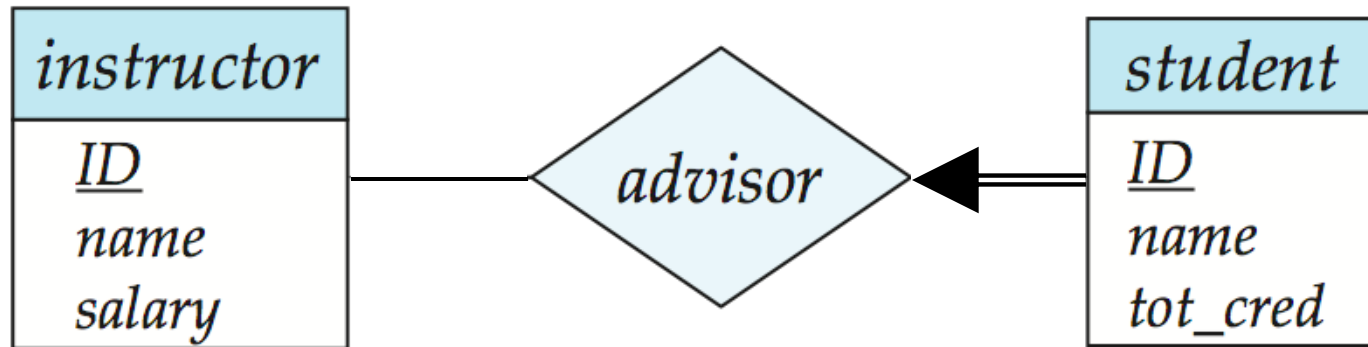
Translation: Examples

- One to many, solution with NULLs



- instructor(ild, name, salary)
- student(sld, name, tot_cred, ild) ild FK ref. instructor
 - If student has no advisor, ild will be NULL

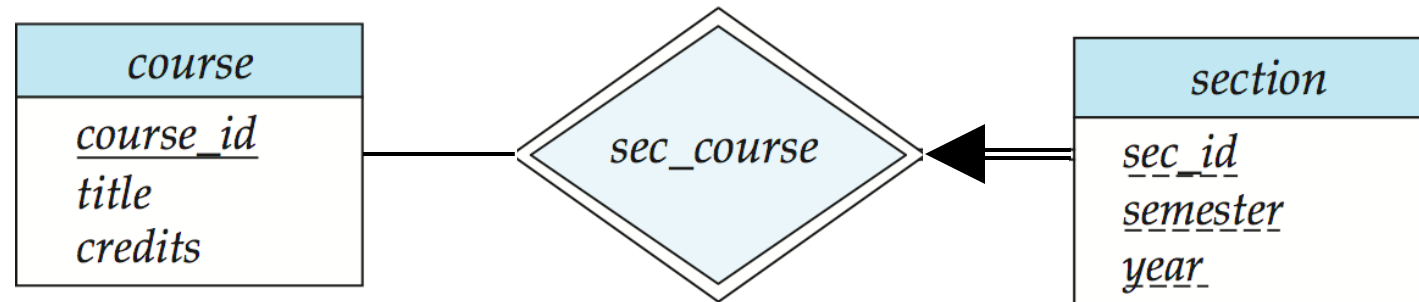
Translation: Examples



- instructor(iID,name,salary)
- student(sID,name,tot_cred,**iID**)

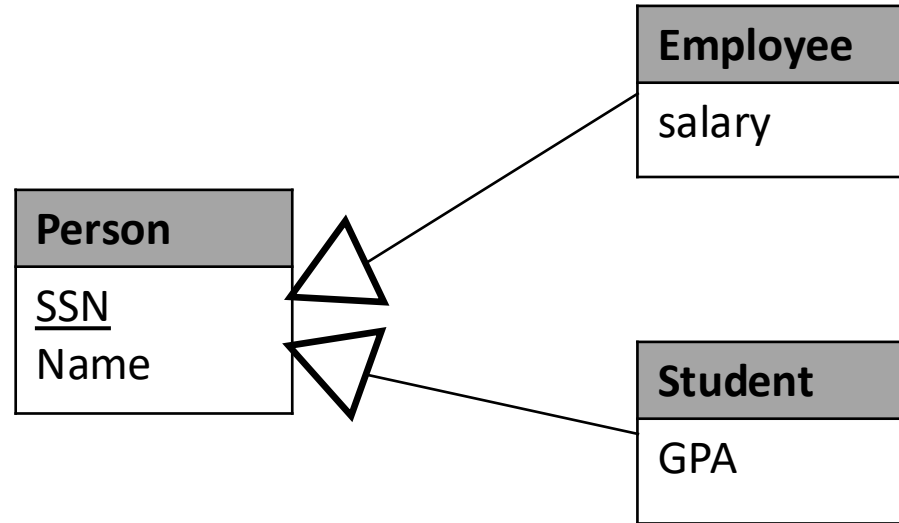
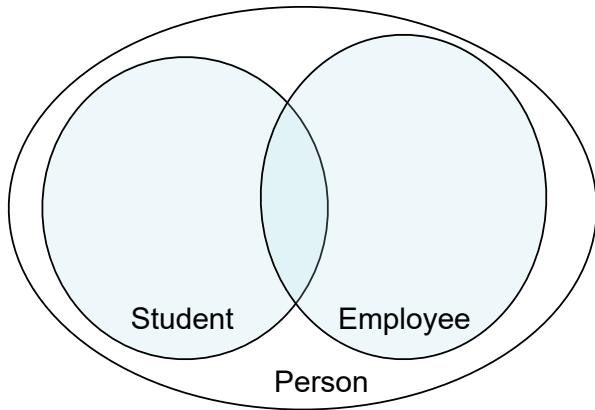
iID FK ref. instructor
iID **not null**

Translation: Examples



- course(course_id, title, credits)
- section(course_id, sec_id, semester, year)
course_id FK ref. course

Translation: Examples



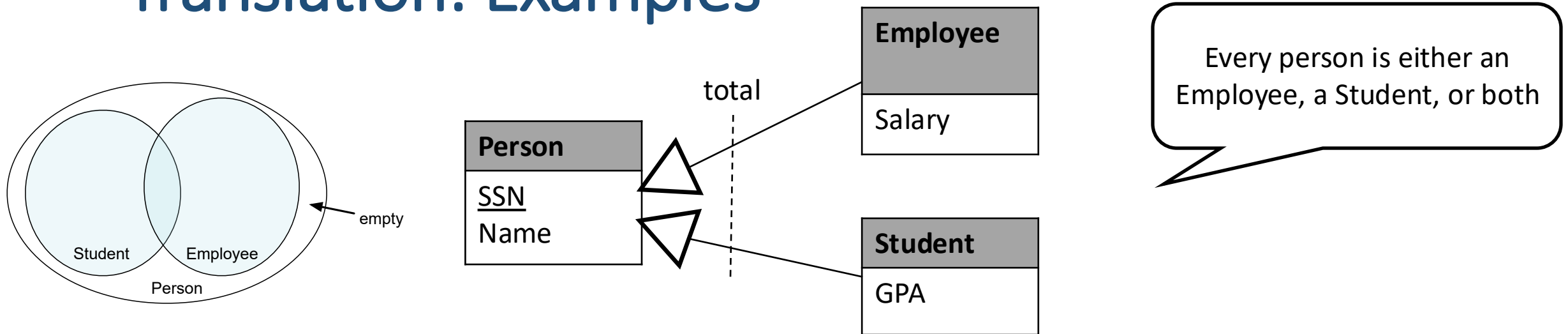
Some persons are also Employee and/or Student

- Person(SSN,Name)
- Employee(SSN,salary)
- Student(SSN,GPA)

SSN FK ref. Person

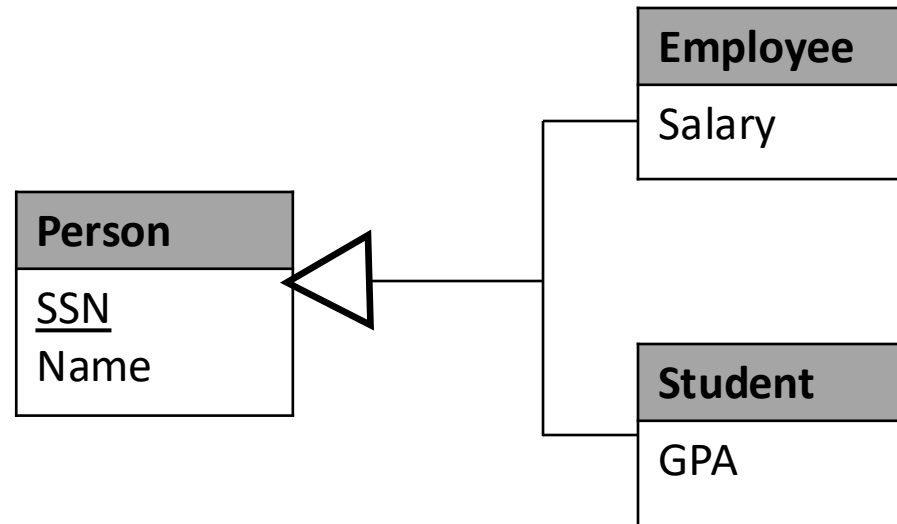
SSN FK ref. Person

Translation: Examples



- Is an example that needs more complex ICs (CHECK see later)
- Basically, need to check that any SSN in Person ref a SNN in Employee or a SNN in Student.
- Safest bet:
- Person(SSN,Name)
- Employee(SSN,salary) SSN FK ref. Person
- Student(SSN,GPA) SSN FK ref. Person

Translation: Examples



Some persons are Employee or Student, but never both

Person(SSN,Name)

Employee(SSN,salary) SSN FK ref. Person

Student(SSN,GPA) SSN FK ref. Person

OR

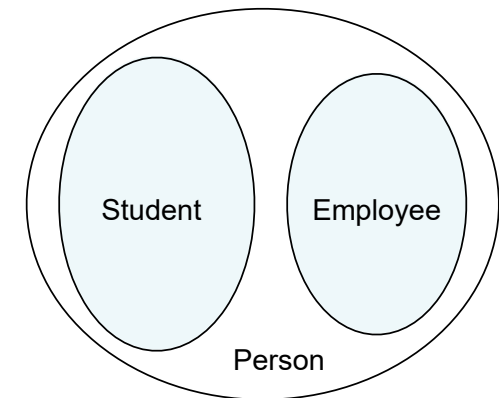
Employee(SSN,name,salary)

Student(SSN,name,GPA)

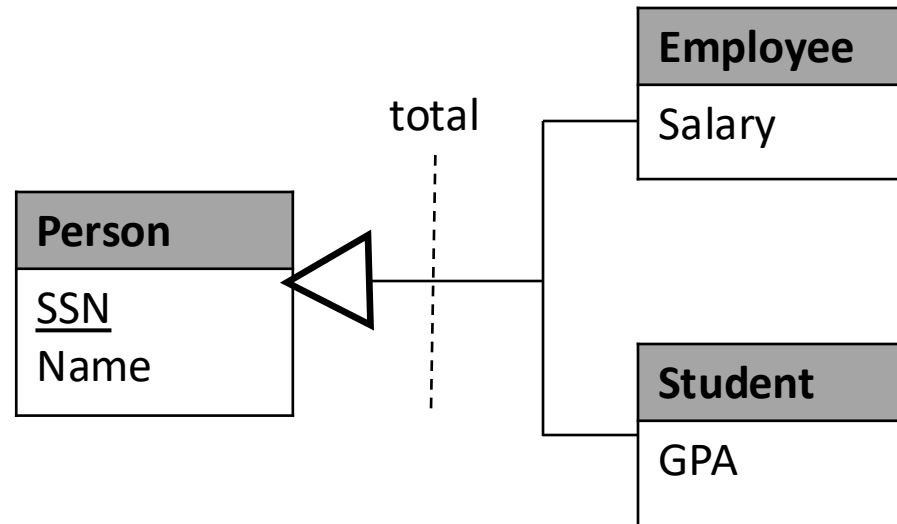
Person(SSN,Name)

CHECK IC needed: No SNN is in both Student and Employee

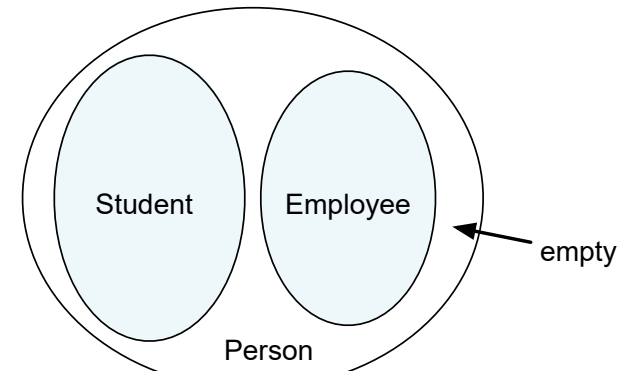
Ok to include name if disjointness holds.



Translation: Examples



Every person is an Employee, or a Student, but never both



Person(SSN,Name)

Employee(SSN,salary) SSN FK ref. Person

Student(SSN,GPA) SSN FK ref. Person

OR

Employee(SSN,name,salary)

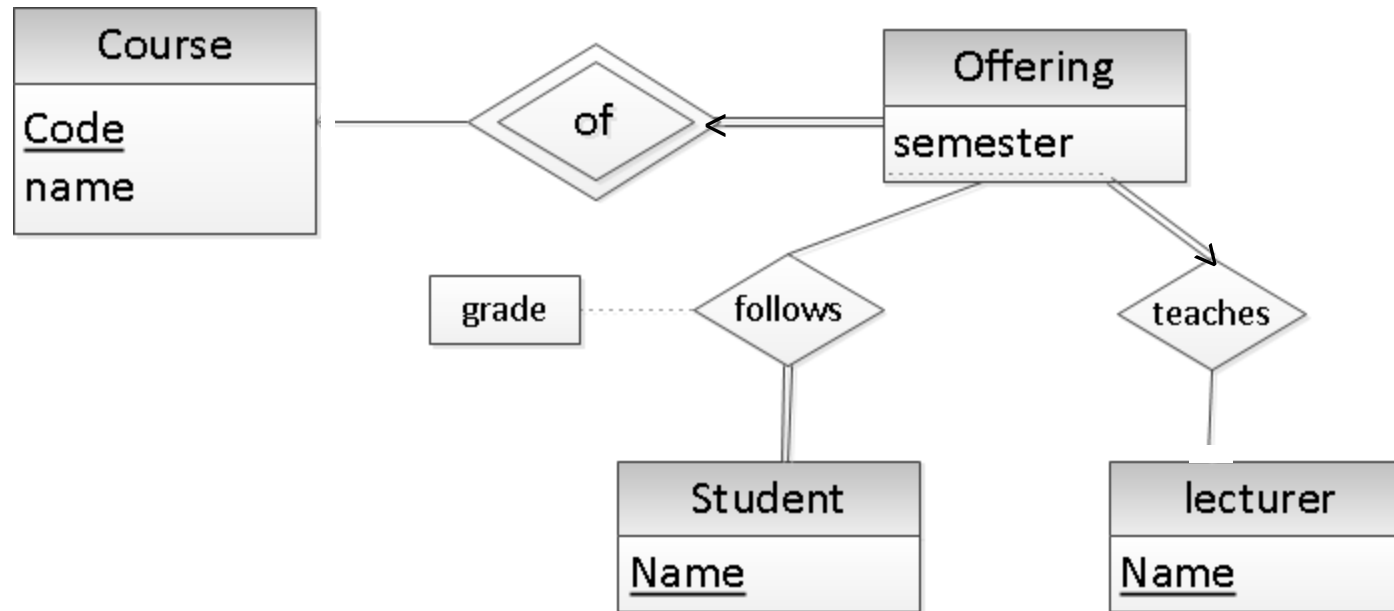
Student(SSN,name,GPA)

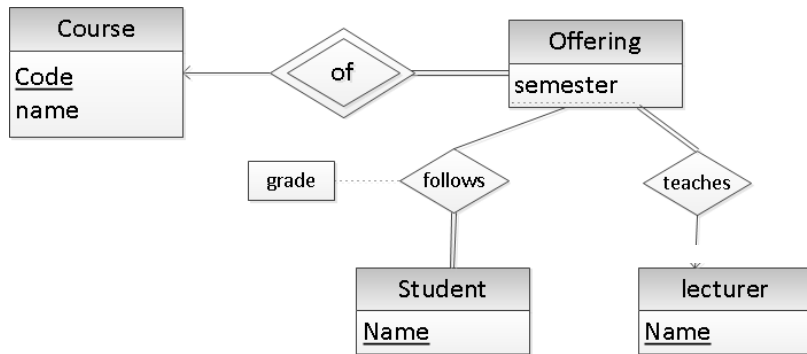
CHECK IC needed: No SNN is in both Student and Employee; no SNN in Person but not in Student or Employee

CHECK IC needed: no SNN in Student and Employee

Exercise: Translate

- Translate the following diagram into the relational model. Mention all primary keys and foreign key constraints.





Course(code,name)

Offering(semester,code,lecturer)

Student(Name)

Lecturer(Name)

Follow(name,semester,code,grade)

Follow.name FK References Student.name

Follow.semester FK References Offering semester

Follow.code FK References Offering code

O.lecturer FK References Lecturer.Name

O.Lecturer NOT NULL

Summary

- Relational model consists of tables plus constraints
 - Type constraints (including NOT NULL)
 - Primary key constraints
 - Foreign key constraints
- We can implement an ER diagram into the relational model
 - Entities become tables
 - Most relations become tables as well
 - Use of constraints in the relational model to enforce cardinality, participation, primary key constraints in the relational model